

NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

PA1 –Mandatory Project Assignment 1

Part1-RTSSP+Part2-SHP

Name: Nicole Arquissandas

Student number: 75026

1.Introduction

This report describes the design and implementation of two protocols developed for PA1: the Real-Time Secure Streaming Protocol (RTSSP) and the Secure Handshake Protocol (SHP).

The initial system, provided as the starting point for the project, consists of a Streaming server that reads movie files encoded in segments containing MPEG4 media frames, and sends them over UDP to a Box, which is a proxy process that receives the stream and forwards it locally to a media player such as VLC. The channel between the server and the box is completely unprotected in the initial implementation: anyone on the network can capture the packets and reconstruct the movie, modify the content without detection, or inject fake packets. The project addresses all of these threats.

The work is divided into two parts. Part 1 designs and implements RTSSP, which secures the traffic over the UDP channel by encapsulating each media frame inside an encrypted and authenticated packet using symmetric cryptography. In this phase, the cryptographic parameters and keys are statically pre-configured and shared between the server and the proxy through configuration files. Part 2 which extends Part1 by introducing SHP, a TLS-inspired handshake protocol executed before streaming begins. SHP dynamically negotiates the cipher suite, performs mutual authentication using certificates and digital signatures, and establishes fresh session keys through an authenticated Elliptic Curve Diffie-Hellman (ECDH) exchange, eliminating the need for any pre shared material. Together, the two parts form a complete secure streaming stack inspired by the layered design of TLS/DTLS, but adapted to real-time connectionless UDP streaming.

2. Part 1- RTSSP

2.1 Protocol Design

2.1.1 Packet Format

The RTSSP packet format is inspired by the TLS Record Layer Protocol (TLS 1.2/1.3) which maps naturally to the requirements of RTSSP. Every RTSSP packet, whether it carries a control message or an encrypted media frame, begins with the same five byte header:

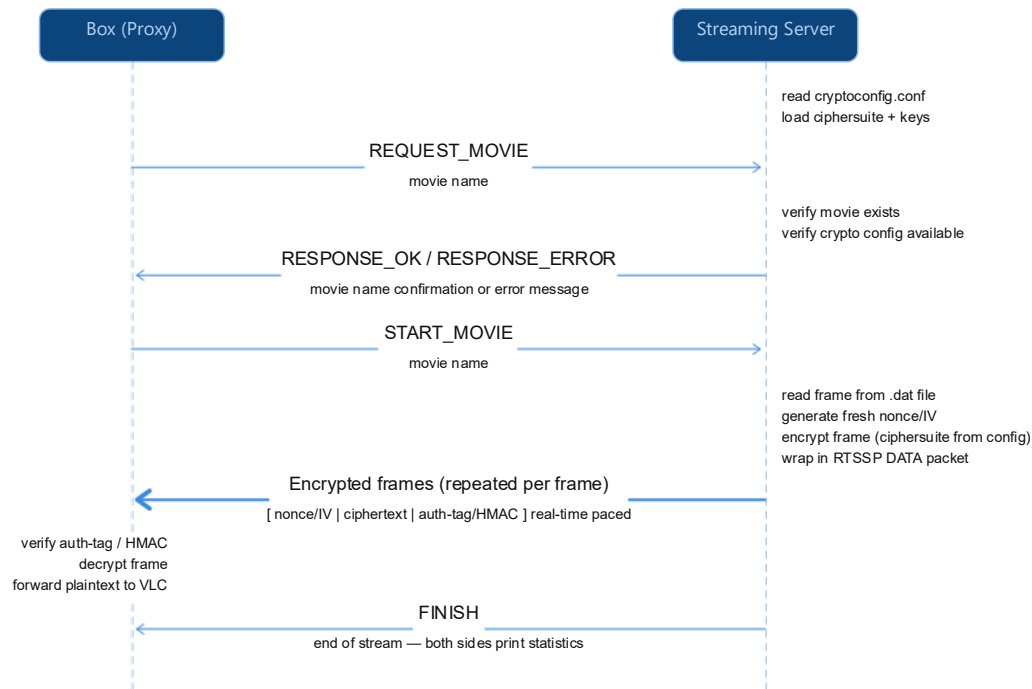
[Content-Type : 1 byte] [Version Major : 1 byte] [Version Minor : 1 byte] [Payload Length Hi : 1 byte] [Payload Length Lo : 1 byte] [Payload : N bytes]

- The Content-Type distinguishes between the two kinds of RTSSP traffic : 0x16 identifies a Control packet and 0x17 identifies a Data Packet(carrying an encrypted media frame) mirroring the TLS handshake content type and data content type.
- The version is fixed 0x01 0x00, representing RTSSP version 1.0
- The payload length is an unsigned 16 bit integer giving the number of bytes that follow the header.
- For CONTROL packets, the first byte of the payload is a subtype identifier that distinguishes the five control message types:
 - REQUEST_MOVIE (0x01), RESPONSE_OK (0x02)
 - RESPONSE_ERROR (0x03)
 - START_MOVIE (0x04)
 - FINISH (0x05).

The remaining bytes of the payload carry a UTF-8 encoded string, which is the movie name for request and response messages, an error description for error messages, and empty for the FINISH message.

- For DATA packets, the payload layout depends on whether the configured cipher suite is AEAD or non-AEAD, and is described in detail in the cryptographic design section below.

2.1.2 Traffic Flow



A complete RTSSP streaming session consists of three phases: session establishment, streaming, and termination. The session establishment phase uses control messages to coordinate the start of streaming. The box initiates by sending `REQUEST_MOVIE` to the server, including the name of the movie it wants. The server looks up the movie file and checks whether a cryptographic configuration exists for it. If both checks pass, the server responds with `RESPONSE_OK` confirming the movie name. If the movie does not exist or has no crypto configuration, the server responds with `RESPONSE_ERROR` containing a description of the problem. Upon receiving `RESPONSE_OK`, the box sends `START_MOVIE`, which is the signal for the server to begin streaming.

During the streaming phase, the server reads the movie file frame by frame. Each frame is stored in the file with a two byte size field, an eight byte timestamp in nanoseconds, and the raw frame bytes. The server uses the timestamps to pace delivery in real time: it computes the elapsed time since the first frame and sleeps for the appropriate duration to ensure each frame is sent at the correct moment. This preserves the original timing of the media content so that VLC can play it smoothly without buffering. Each frame is encrypted and sent as a separate RTSSP DATA packet.

When the last frame has been sent, the server sends a `FINISH` control message to the box. Both sides then compute and print statistics: the total number of frames sent or received

and the throughput in frames per second, as specified by the project requirements. The session ends after FINISH is sent.

2.2 Cryptografic Design

2.2.1 Configuration

The cryptographic parameters for RTSSP are defined in a plaintext configuration file called `cryptoconfig.conf`, which must be identical on both the server and the box. The file maps each movie to a cipher suite, an encryption key, and optionally an HMAC algorithm and mackey (For non-AEAD cipher suites that require a separate MAC). Example configuration entry looks like this:

```
<monsters.dat.encrypted>  
ciphersuite: AES/CTR/NoPadding  
key: <256-bit hex key>  
hmac: HmacSHA256  
mackey: <256-bit hex key>  
</monsters.dat.encrypted>
```

```
<cars.dat.encrypted>  
ciphersuite: AES/GCM/NoPadding  
key: <256-bit key in hexadecimal>  
</cars.dat.encrypted>
```

The server reads this file once at startup. . No dynamic reload is performed, consistent with the project specification, which describes `cryptoconfig.conf` as a manual and static cryptographic configuration.

2.2.2 Supported Cipher Suites

The implementation supports four cipher suites: two are AEAD (Authenticated Encryption with Associated Data) and two are non-AEAD with a separate HMAC for integrity. The table below summarises the supported suites and their Data payload formats:

Cipher Suite	Category	Key Size	IV/Nonce	DATA Payload Format
AES/GCM/NoPadding	AEAD	128/192/256-bit	12 bytes	nonce(12) ciphertext tag(16)
ChaCha20-Poly1305	AEAD	256 bit	12 bytes	nonce(12) ciphertext tag(16)
AES/CTR/NoPadding + HMAC-SHA256	Non-AEAD	128/192/256-bit	16 bytes	iv(16) ciphertext hmac(32)
AES/CBC/PKCS5Padding + HMAC-SHA256	Non-AEAD	128/192/256-bit	16 bytes	iv(16) ciphertext hmac(32)

2.2.3 AED: AES-GCM and ChaCha20-Poly1305

For AEAD ciphers, a fresh 12-byte nonce is generated using SecureRandom for every frame. The nonce is prepended to the ciphertext, which already includes a 16-byte authentication tag appended automatically by the JCA. On the receiving side, the nonce is extracted from the first 12 bytes, and the JCA verifies the authentication tag during decryption, rejecting any tampered packet before the plaintext is ever returned.

2.2.4 Non-AED: AES-CTR and AES-CBC

For non-AEAD ciphers, the implementation uses Encrypt-then-MAC (EtM). A fresh 16-byte IV is generated per frame. After encryption, HMAC-SHA256 is computed over the concatenation of the IV and the ciphertext, and appended to the packet. On decryption, the HMAC is verified first before decryption begins so that any modification to the ciphertext is detected before any decryption attempt is made. If verification fails, a SecurityException is thrown and the frame is discarded. Confidentiality-only configurations are explicitly rejected at startup.

2.3 Security properties

The RTSSP implementation satisfies all four security properties required by the project specification:

- Connectionless confidentiality is provided because every media frame is encrypted before transmission, and an attacker who captures UDP packets cannot recover the plaintext without the symmetric key. The use of a fresh IV (generated by using Java's SecureRandom) per frame ensures that even if the same frame appeared twice in the movie, the two packets would produce different ciphertexts, preventing traffic analysis based on ciphertext patterns.
- Connectionless integrity without recovery is provided by the AEAD authentication tag or the HMAC. Any modification to a single bit of the ciphertext or the IV will cause verification to fail on the receiving side, and the frame will be silently discarded. There is no retransmission since the protocol is real time.
- Data-origin authentication follows from the same mechanism as integrity: the authentication tag or HMAC can only be produced by someone that knows the symmetric key, which is shared exclusively between the server and the box through the cryptoconfig.conf file. An attacker who does not have the key cannot produce a valid tag and therefore cannot inject authenticated packets into the stream.
- Replay protection is provided each frame carries a fresh random nonce or IV. A replayed packet will attempt decryption with the same nonce under the same key, which for GCM would produce a tag mismatch if the attacker modifies any byte, and for CTR/CBC the HMAC would fail on any modified ciphertext.

2.4 Implementation

The implementation consists of six Java classes:

- RTSSPPacket.java - defines packet constants, builds and parses RTSSP packets
- CryptoConfig.java - data class holding the cipher suite, key, and optional HMAC parameters for a movie

- CryptoConfigParser.java -parses cryptoconfig.conf and maps movie names to CryptoConfig instances
- CryptoUtils.java -handles encrypt and decrypt for all four cipher suites, dispatching to AEAD or non-AEAD paths
- RTSSPServer.java -listens for control messages, handles requests, and streams encrypted frames
- RTSSPProxy.java -requests a movie, receives the encrypted stream, decrypts each frame, and forwards it to VLC

3. Part 2- SHP

In this phase Part 2 we implement SHP, a handshake protocol that runs before RTSSP streaming begins. SHP has two purposes: mutual authentication, meaning both sides verify each other's identity before any session keys are established, and dynamic key establishment, meaning the session keys are derived fresh for every session through a cryptographic key exchange and never stored anywhere or transmitted over the network. The cipher suite is negotiated during the handshake rather than hardcoded in a file. The design is inspired by the TLS 1.3 handshake protocol, adapted for UDP and for the specific requirements of the streaming system.

3.1 Cryptographic Design

Before describing the protocol messages, it is necessary to explain the three cryptographic mechanisms that SHP relies on.

3.1.1 Long-term EC Keys and Certificates – Authentication

Each entity holds a long-term EC key pair on curve secp256r1, stored in a PKCS12 keystore generated with keytool (the same approach used in Lab 3). The private key is used exclusively for ECDSA signatures to prove identity. The corresponding certificate is included in handshake messages so the other side can verify the signature.

Each side maintains a truststore containing the certificate of the other party.

Authentication is performed by checking whether the received certificate is present in the local truststore, and then verifying the ECDSA signature over the message body using the public key from that certificate.

3.1.2 Ephemeral ECDH Keys - Key Exchange

In addition to their long-term keys, both the server and the box generate a fresh ephemeral EC key pair at the start of every handshake session. These are temporary keys that exist only in memory for the duration of the handshake. The public keys are exchanged in CLIENT_HELLO and SERVER_HELLO (shown in Protocol design section below). The EC private keys never leave their respective machines. Both sides then compute the ECDH shared secret independently.

This provides forward secrecy because after the session ends, both parties discard their ephemeral private keys. If an attacker records the encrypted stream and later compromises the long-term private keys, they still cannot recover the ephemeral private keys and therefore cannot reconstruct the ECDH shared secret or the derived session keys. Past sessions remain protected even after a long-term key compromise. This is the primary security advantage of Part 2 over Part 1 where the same symmetric keys are reused for every session and are stored permanently in cryptoconfig.conf. If an attacker obtains that file, every session ever recorded becomes decryptable. In Part 2, each session uses a unique set of keys that exist nowhere after the session ends.

3.1.3 HKDF-SHA256 (inspired by TLS 1.3) - Key Derivation

The raw ECDH shared secret is not used directly as a session key. Instead, it is used into an HMAC-based Key Derivation Function ,because a single shared secret needs to produce multiple independent keys, one for encryption and one for MAC in the case of non-AEAD suites, and using the same secret for both would create a dependency between them.

The implementation of HMAC-based Key Derivation Function uses a two-step HKDF-SHA256 construction. In the Extract step, HMAC-SHA256 is computed with the salt set to the concatenation of the client nonce and the server nonce, and the input key material set to the ECDH shared secret. This produces a pseudorandom key called PRK. In the Expand step, independent output keys are derived by computing HMAC-SHA256 of PRK concatenated with a domain separation label and a counter byte:

```
PRK  = HMAC-SHA256( clientNonce || serverNonce , sharedSecret )  
  
encKey = HMAC-SHA256( PRK , "RTSSP-ENC-KEY" || 0x01 ) [first 32 bytes]  
  
macKey = HMAC-SHA256( PRK , "RTSSP-MAC-KEY" || 0x02 ) [first 32 bytes]
```

Mixing both nonces into the PRK is important for key freshness. Even if the same ECDH public keys were reused across sessions, the nonces would be different each time, producing different PRK values and therefore different session keys. The domain separation labels (RTSSP-ENC-KEY, RTSSP-MAC-KEY) ensure that the encKey and the macKey are cryptographically independent even though they come from the same PRK.

3.2 Protocol Design

3.2.1 SHP Message Format

SHP messages share a common outer structure. Each field within the message body is serialised using a 4-byte big-endian length prefix followed by the field bytes. This is the same framing approach used in TLS record format.

CLIENT_HELLO and SERVER_HELLO Outer Message Structure

Field	Type	Size	Description
type	byte (1)	1 byte	Message type: 0x01=CLIENT_HELLO, 0x02=SERVER_HELLO, 0x03=CSSP
bodyLen	int32 (4)	4 bytes	Length of the body in bytes, big-endian
body	byte[]	bodyLen bytes	Serialised message fields (see per-message tables below)
signatureLen	int32 (4)	4 bytes	Length of the ECDSA signature in bytes
signature	byte[]	signatureLen bytes	ECDSA signature over body bytes using sender's long-term private key. Not present in CSSP (payload is encrypted instead)

CSSP Outer Structure

Field	Type	Size	Description
type	byte (1)	1 byte	0x03 = TYPE_CSSP
payloadLen	int32 (4)	4 bytes	Length of the encrypted payload
encryptedPayload	byte[]	payloadLen bytes	Encrypted with derived encKey: { nonceResponse(byte[]) 'READY'(UTF-8) }

3.2.2 SHP CLIENT_HELLO

The box initiates the handshake by sending CLIENT_HELLO to the server. The message body contains:

- The name of the requested movie
- The box ephemeral ECDH public key (so the server can compute the shared secret)
- The box certificate (so the server can verify the box's identity)

- list of proposed cipher suites
- A 32-byte random nonce (challenge to the server generated with secureRandom)

The entire CLIENT_HELLO body is then signed with the box's long-term private key. The signature is appended to the message.

CLIENT_HELLO Body Structure

Field	Type	Size	Description
movieName	UTF-8 String	4 + N bytes	Length-prefixed UTF-8 string with the requested movie filename
ecdhPublicKey	byte[]	4 + 91 bytes	X.509 encoded ephemeral EC public key (secp256r1)
certificate	byte[]	4 + N bytes	DER-encoded X.509 box certificate from box.p12 keystore
ciphersuites	UTF-8 String	4 + N bytes	Comma separated list of proposed cipher suites in preference order
clientNonce	byte[]	4 + 32 bytes	32 random bytes generated with SecureRandom - challenge to the server

3.2.3 SHP SERVER_HELLO

When the server receives CLIENT_HELLO, it does not immediately respond, It first performs three verification steps before constructing its response. First, it checks whether the certificate included in the message is present in the server truststore. If it is not, the connection is rejected immediately. Second, it verifies the ECDSA signature over the CLIENT_HELLO body using the public key extracted from the box certificate. A valid signature proves both that the message came from the box and that it was not tampered with. Third, it verifies that the requested movie file exists and that a crypto configuration is available for it.

If all checks pass, the server selects the cipher suite. The server has a single preferred cipher suite configured in server.properties. It checks whether the box's proposed list includes that suite. If yes, the selected suite is confirmed. If no, the handshake is rejected because no common suite exists.

The server then generates its own ephemeral ECDH key pair and a 32 byte server nonce, and computes the nonce response: an ECDSA signature over the client nonce using the server's long-term private key. This signature is the server's answer to the box's challenge, proving to the box that the server owns its private key.

After that the server constructs The SERVER_HELLO message and sends it to the box. The body of the message contains:

- Confirmation of the requested movie name
- The server ephemeral ECDH public key
- The server certificate
- The selected cipher suite (chosen from the client proposal)
- A 32-byte random nonce (challenge to the box)
- A response to the box nonce: ECDSA signature of the client nonce using the server private key

The entire body is signed with the server's long-term private key and the signature is appended to the message.

After this exchange both sides have everything they need to independently derive the session keys without communicating further. The box has its own ephemeral private key and the server ephemeral public key, so it can compute the ECDH shared secret. The server has its own ephemeral private key and the box ephemeral public key, so it can compute the same shared secret. Both also have the client nonce and server nonce, which are mixed into HKDF along with the shared secret to produce the final session keys.

SERVER_HELLO Body Structure

Field	Type	Size	Description
movieName	UTF-8 String	4 + N bytes	Confirmation of the requested movie filename
ecdhPublicKey	byte[]	4 + 91 bytes	X.509 encoded ephemeral EC public key (secp256r1)
certificate	byte[]	4 + N bytes	DER-encoded X.509 server certificate from server.p12 keystore
selectedSuite	UTF-8 String	4 + N bytes	The single cipher suite selected by the server from the client proposal
serverNonce	byte[]	4 + 32 bytes	32 random bytes generated with SecureRandom -challenge to the box
nonceResponse	byte[]	4 + N bytes	ECDSA signature of clientNonce using server long-term private key

3.2.4 SHP CSSP

After receiving and validating SERVER_HELLO, the box also performs its verification steps. It checks the server certificate against its truststore, verifies the SERVER_HELLO body signature, and verifies the nonce response: it confirms that the signature over the client

nonce was produced by the private key corresponding to the server certificate. If all three checks pass, the box derives the session keys using ECDH and HKDF exactly as described above.

The box then constructs CSSP, the final handshake message. The plaintext payload of CSSP contains two items:

- A response to the server nonce: ECDSA signature of the server nonce using the box private key
- The string READY, signalling the box is ready to receive the stream

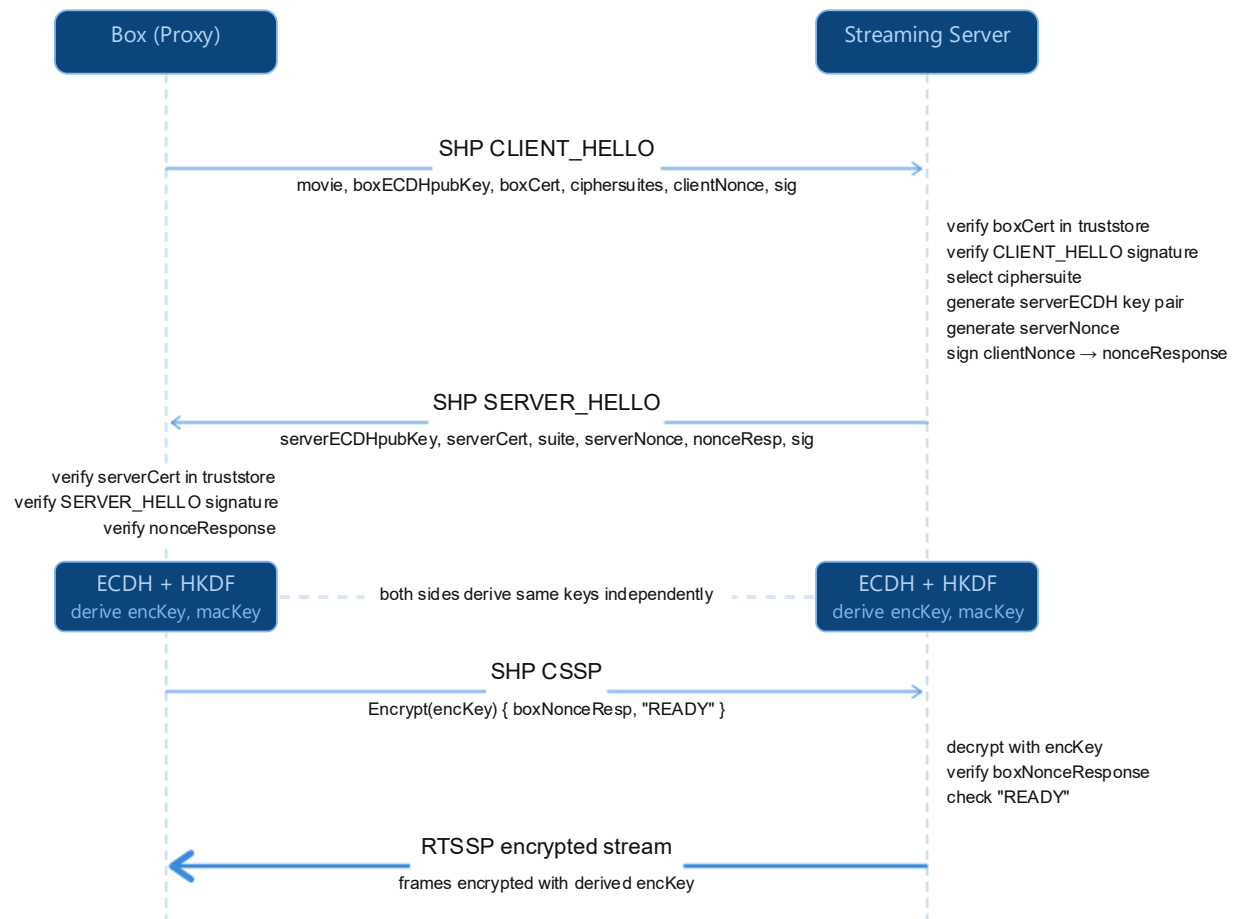
This plaintext is encrypted using the derived session keys. CSSP is therefore the first message protected by the session keys.

When the server receives CSSP, it decrypts it using the session keys it derived independently. Successful decryption is the first confirmation that both sides derived the same keys. The server then verifies the box's nonce response: it checks that the signature over the server nonce was produced by the box private key. If this is valid, and if the READY string is present, the handshake is complete and RTSSP streaming begins using the derived session keys.

CSSP Plaintext Payload (before encryption)

Field	Type	Size	Description
nonceResponse	byte[]	4 + N bytes	ECDSA signature of serverNonce using box long-term private key
ready	UTF-8 String	4 + 5 bytes	The string 'READY' signalling the box is ready to receive the stream

3.2.5 Full Handshake Flow



3.3 Security Properties

The following table summarises the key properties and how each is achieved:

Property	How it is achieved
Mutual authentication	Both sides present their certificate and prove ownership of the corresponding private key through ECDSA signatures. The server verifies the box certificate and signature in <code>CLIENT_HELLO</code> . The box verifies the server certificate and signature in <code>SERVER_HELLO</code> . Neither side can

	impersonate the other without access to the legitimate private key.
Challenge-response	Each side sends a random nonce as a challenge. The other side must sign it with their private key and return the signature. The box verifies the server nonce response in SERVER_HELLO; the server verifies the box nonce response in CSSP. This prevents replay attacks where an attacker records a previous handshake and plays it back.
Forward secrecy	Ephemeral ECDH key pairs are generated fresh for every session and discarded after use. An attacker who later compromises the long-term private keys of either party cannot reconstruct the ephemeral keys and therefore cannot derive the session keys or decrypt any recorded session.
Key freshness	Both the client nonce and the server nonce are mixed into the HKDF salt. Even if the same ECDH parameters appeared in two sessions, the nonces would produce different PRK values and therefore different session keys.
No key transmission	The session keys are never transmitted. Both sides derive them independently from the ECDH shared secret and both nonces. An attacker who eavesdrops the entire handshake cannot compute the shared secret without knowing one of the ephemeral private keys.
Key confirmation	CSSP is encrypted with the derived keys. The server decrypting CSSP successfully is implicit confirmation that both sides computed the same session keys.
Confidentiality and integrity	All RTSP frames are protected by the derived session keys using the same AEAD or non-AEAD (which uses the Encrypt then mac method) constructions.

3.4 Implementation

Part 2 adds four Java classes on top of the Part 1 classes:

- SHPMessage.java -Handles binary serialization and deserialization of the three handshake messages using 4 byte length prefixed fields
- SHPHandshakeUtils.java – Encapsules all asymmetric cryptographic operations: ECDH key generation, ECDH shared secret computation, HKDF-SHA256 key derivation, ECDSA signing and verification, and PKCS12 keystore loading
- SHPRTSSPServer.java - The main server class that handles incoming SHP handshakes and starts the RTSSP stream after a successful handshake. Since the server uses a single shared UDP socket, a routing mechanism was implemented using BlockingQueues to ensure that each handshake thread receives the correct packets, avoiding race conditions between the main receive loop and the handshake threads.
- SHPRTSSPProxy.java - performs the full three message SHP handshake and then receives the RTSSP stream using the derived session keys

The cipher suite is configured in server.properties via a single ciphersuite property. The server selects the configured suite if the box supports it otherwise the handshake is rejected.